

TDDE70 Deep Learning – Project Description

Fine-Tuning CLIP for Remote Sensing Data

1 Project Overview

The goal of this project is to get hands-on experience with fine-tuning a large vision-language model (CLIP) on remote sensing data, i.e., images collected from satellites and aircraft. Specifically, you will work with the RSICD dataset, which is a remote sensing image captioning dataset with images collected from Google Earth, Baidu Map, MapABC, and Tianditu. Through this project, you will learn how to adapt a pre-trained CLIP model to a specific domain and evaluate its performance in several relevant tasks, such as zero-shot classification and retrieval.

In addition to fully-supervised fine-tuning, this project also encourages exploring semi-supervised fine-tuning, where both labeled and unlabeled data are incorporated into the training process.

2 Prerequisites

To succeed in this project, all project members should have:

- A solid understanding of deep learning concepts.
- Experience with Python and PyTorch.
- Experience with using a GPU-enabled computing environment for training.

3 Data

The dataset for this project is RSICD¹, which can be downloaded from the RSICD GitHub repository (requires Git LFS) or from the project folder in Lisam. You need to download three files from the repository:

¹https://github.com/201528014227051/RSICD_optimal

Table 1: Basic information about the RSICD dataset.

<code>RSICD_images</code>	Folder containing all images in the dataset.
<code>txtclasses_rsicd</code>	Text file containing the class names and their corresponding image file names.
<code>dataset_rsicd.json</code>	JSON file describing each image, including captions and split information (train/val/test).

Dataset split

The dataset is originally divided into three subsets for training, validation, and testing, defined in `dataset_rsicd.json` under the “split” tag. However, we will use the available data in a slightly different way in this project, and therefore define the following three data splits:

- **Split A** (referred to as “train” in the json file; 8734 images) – This will be treated as the **unlabeled** dataset for semi-supervised fine-tuning. **Only images, no captions, can be used in this project!**
- **Split B** (referred to as “validation” in the json file; 1094 images) – This will be treated as the labeled dataset for training (images with captions). It is suggested to further split this into “real” training and validation sets.
- **Split C** (referred to as “test” in the json file; 1093 images) – This will be treated as the labeled dataset for final evaluation (images with captions and class labels).

Note that each image is accompanied by five text descriptions (captions). It is recommended to utilize all of the captions from Split B during training.

4 Models

The project will use OpenCLIP², an open-source implementation of CLIP.

Basic model

ViT-B/32: This is the default and basic model for the main experiments.

Optional exploration

If you are interested in exploring other CLIP models, you are welcome to do so and can include such results in your report.

²https://github.com/mlfoundations/open_clip

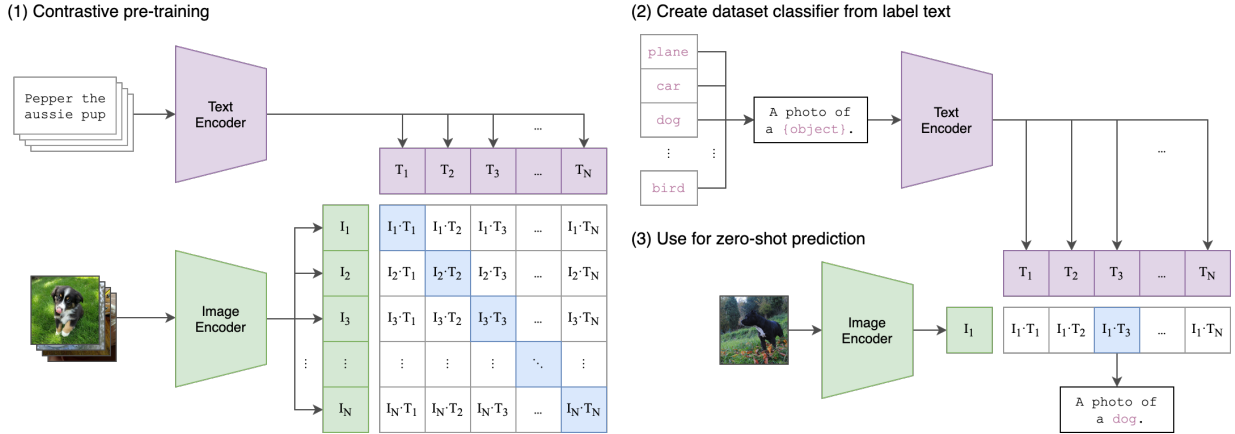


Figure 1: CLIP model overview. Image credit: <https://github.com/openai/CLIP>

5 Tasks and Experimental Setup

Project settings

This project has two difficulty levels:

Table 2: Project versions.

Basic version	Fully-supervised fine-tuning using only the labeled dataset with images + captions for training (Split B). Performance is tested on the test set (Split C).
Advanced version	Semi-supervised fine-tuning using both the labeled dataset (Split B) and the unlabeled dataset with only images (Split A). You need to design and implement appropriate loss functions to leverage both types of data. Performance is tested on the test set (Split C).

Note that you are **not allowed** to use the test set (Split C) for training or tuning of hyperparameters in either of the two versions.

Suggested evaluation tasks on the test set

At a minimum, all students should implement zero-shot classification on the test set. For the advanced version of the project, a few additional tasks of your choice, like image-to-text retrieval, text-to-image retrieval, or other creative extensions related to the project, should also be implemented (see Section 7: Grading Criteria).

- **Zero-shot classification [mandatory]** means that a model can classify images into categories without being explicitly trained as a classifier. Specifically, the CLIP model

can use textual descriptions to assign labels for a given remote sensing image. For example, define text-based descriptions for all categories in this project according to the `txtclasses_rsid` file:

```
"A satellite image of {category A}"
"A satellite image of {category B}"
...
"A satellite image of {category X}"
```

CLIP computes similarity scores between the input image embedding and each text embedding. The category with the highest similarity score is assigned as the predicted label.

- **Zero-shot classification with multiple templates** builds on the idea above but considers many textual descriptions of the same category. That is, instead of just using "A satellite image of {}" as a template, you can define new templates and evaluate the similarity scores for all of them. It is up to you to decide how the scores should be combined in a good way.
- **Image-to-text retrieval** aims to retrieve the most relevant textual descriptions for a given image. Note that in this case, the text descriptions from the `dataset_rsid.json` file corresponding to each input image will serve as the ground truth.
- **Text-to-image retrieval** aims to retrieve the most relevant image for a text query. Similar to image-to-text retrieval, the text descriptions from the `dataset_rsid.json` file will serve as the text query when evaluating the model, so that we can associate each query with a correct image. However, a qualitative analysis of arbitrary text queries can also be performed.
- As mentioned above, feel free to define other relevant tasks and test the model on these.

Instructions for using OpenCLIP base code

You have flexibility in choosing how to utilize OpenCLIP in your project. The official OpenCLIP GitHub repository provides a comprehensive tutorial for easy implementation (using `open_clip` as a package), which is recommended for a straightforward approach. If you need more flexibility, specifically for the Advanced version of the project, direct modification of the source code is also encouraged to allow for more customization.

Note that the main focus for the Advanced version should be on developing novel ideas for semi-supervised learning to effectively fine-tune the CLIP model.

Automation requirement

Any data processing, augmentation, or modification of captions must be implemented through automated procedures. Manual editing of captions or labels for individual samples is **not allowed**.

For example, if caption augmentation or refinement is performed, it should be implemented programmatically (e.g., using scripts, data processing pipelines, or API calls to external models). The goal is to ensure that the proposed methods are reproducible and methodologically motivated, rather than relying on manual tuning of individual examples. Furthermore, we want you to focus on model development and implementation, not on manually tweaking the data.

6 Instructions for the Final Report

The final report should be a self-contained document formatted according to the NeurIPS paper template³ and submitted as a single PDF through Lisam. The main section of the report is limited to 6 pages (excluding references, ethical considerations, and contribution statements).

The main part of the report must include the following sections:

- **Abstract:** A summary of the project within 150 words. It should concisely explain the problem you aim to solve, your ideas, and how the results reflect your methods.
- **Introduction:** Background and motivation for this study. This is a high-level explanation and motivation of the problem formulation. The introduction should also contain key references to relevant literature. An exhaustive literature review is not required for the project, but any related work that you take inspiration from and/or base your solution on should of course be cited and discussed.
- **Problem Formulation:** Formal definition of the problem being addressed, written in a concise but clear and unambiguous mathematical language. For instance, if you are solving a classification problem, then you should clarify the goal of the classification in the language of probability theory. Here, you should clearly state whether you target the basic or the advanced version of the project, and write your problem formulation accordingly.
- **Method:** Description of the approach and techniques used.
- **Results:** Presentation and analysis of experimental results.

³<https://media.neurips.cc/Conferences/NeurIPS2024/Styles.zip>

- **Conclusion/Discussion:** Summary of insights and potential future work.

In addition to the above, the report should contain the following, not counted towards the 6-page limit:

- An ethical considerations section to reflect on any ethical implications of the work.
- A contribution statement which clearly specifies each group member's individual contributions. This statement should be sufficiently detailed for the examiner to assess each team member individually. A too vague statement could thus result in a lower grade due to unclarities about how you contributed to the work. To facilitate meaningful collaboration, each group member is expected to take **primary responsibility for one or more major components** of the project, such as: model implementation or modification, training pipeline development, data preprocessing or augmentation, design of semi-supervised learning strategies, implementation of additional evaluation tasks (e.g., retrieval or template-based classification), experiment design and hyperparameter tuning, result analysis and visualization, leaderboard submission pipeline, or report writing and documentation. The contribution statement should also explain if, and in that case how, you have used AI tools for developing your models, writing code, and writing the final report.
- Optionally, up to 5 additional pages of appendices for supplementary material (e.g., detailed derivations, extra supporting results), but the main part of the report must be self-contained (i.e., it should be possible to read, understand, and grade the report without the supplementary materials).

7 Grading Criteria

The project will be graded on a U/3/5 scale, with the following criteria:

■ 3 (Pass)

You should achieve the following points.

1. Fully-supervised fine-tuned CLIP and evaluated on zero-shot classification on the test set (basic version of project).
2. Evaluation, analysis, and critical assessment of the results, using good machine learning practices.
3. Clear and solid report as per the instructions provided.

Furthermore, a passing grade requires active participation in the project seminar; see Section 9.

■ 5 (Pass with distinction)

You should achieve the following points.

1. Effective semi-supervised fine-tuning using both labeled and unlabeled data (advanced version of project), along with at least three evaluation tasks.
2. Comprehensive evaluation, analysis, and critical assessment of the results, using good machine learning practices. Some form of ablation studies⁴, parameter analysis⁵, and/or comparisons between different models should be included (but you may not have the time or computational resources to cover all of these aspects).
3. Active participation on the leaderboard (see Section 8).
4. Clear and solid report as per the instructions provided. **We will put particular emphasis on the clarity of the mathematical formulation of the problem and the chosen approach.**
5. Presentation of the project at the project seminar (see Section 9). For groups aiming for grade 5, the seminar presentation will serve as an oral examination of the advanced components of the project. Students should therefore be able to explain and justify their methodological choices, experimental setup, and results.

Note that grades are set individually and it is possible that different members of a certain project group will get different grades. Specifically, this means that in order to get grade 5 you need to: (i) clarify in the contribution statement that your individual contribution is sufficient to meet the criteria above, and (ii) actively participate in the project seminar presentation and be prepared to answer questions about the work.

8 Leaderboard

To promote friendly competition and evaluation of your models in a standardized way, we will maintain a leaderboard based on your zero-shot classification performance on a hidden test set. Note that the leaderboard is optional if you only aim for grade 3, but mandatory if you aim for grade 5. **In either case, your grade is completely independent of**

⁴Evaluating the impact of specific components by removing or modifying them.

⁵Evaluating how different hyperparameters affect performance.

your performance on the leaderboard.⁶ However, we still encourage everyone to join the competition, as it can be a fun experience and a valuable addition to your portfolio and future applications.

Download the data

We will use a new dataset where the true labels are not accessible to you. Download the dataset (`data4leaderboard.zip`) from the project folder in Lisam. Alternatively, you can also download it in the leaderboard platform (see the CodaBench instructions). Unzip the file to get a folder of 2,100 images, each of size $256 \times 256 \times 3$ pixels.

Target classes

There are 21 land use categories in total:

agricultural, airplane, baseballdiamond, beach, buildings,
chaparral, denseresidential, forest, freeway, golfcourse,
harbor, intersection, mediumresidential, mobilehomepark, overpass,
parkinglot, river, runway, sparseresidential, storagetanks, tenniscourt

You should use these class names as your label space.

Prepare your submission

Based on the input images, your model should generate corresponding label predictions. You should enclose all the label predictions in a `predictions.txt` file using the following format:

```
image_filename predicted_class
image_filename predicted_class
...
```

Please ensure one line per image. `image_filename` must match the file name exactly, including the file extension (e.g., `ak29dkfj8s.jpg`). `predicted_class` must be one of the 21 valid class names listed above. Ensure that every image in the test set is included—missing even a single entry will result in 0 accuracy. Also ensure that the list has no repetition.

As a side note, the ordering of the entries in the list does not matter.

Example `predictions.txt`:

⁶The reason why participation in the leaderboard is mandatory for the higher grade is that it is intended to be a motivating and fun way to compare innovative solutions from the advanced version of the project.

Final

?

Number of submissions used for the day

1 out of 2

Number of total submissions used

1 out of 10

Submission upload

Submit as: ?

Yourself

▶ Running predictions.zip (ID -)

Search...

Status

ID #	File name	Date	Status	Score	Actions
-	predictions.zip	2025-04-04 13:44	Finished		

Detailed results (e.g., confusion matrix) are shown here.

Click this to send your results to the leaderboard

Figure 2: A demo for how to use the CodaBench leaderboard.

a8s7f1k2jd.jpg river
 1kd02ks9dj.jpg airplane
 ...

How to use the CodaBench leaderboard

First, go to [the CodaBench competition page](#). Navigate to “My Submissions” and register for the competition.

N.B. Participation on the leaderboard according to the instructions above requires an account on CodaBench. If you prefer not to register on this third-party platform but still want to participate in the competition (e.g., for fulfilling the requirement for the higher grade), you may upload your `predictions.txt` file using the submission function in Lisam instead. This will however limit your participation to a single submission (the solution will then be submitted to CodaBench after the deadline) and you will thus not get continuous feedback on how well you are doing. Submitting your solution to Lisam is only necessary if you do not submit to CodaBench by yourself.

Submission and evaluation

1. Double-check if your `predictions.txt` file is correctly formatted as per the instructions.
2. Zip `predictions.txt` as `predictions.zip`.
3. Navigate to “My Submissions” in the competition page in CodaBench. Upload your `predictions.zip` (not `.txt`) and wait for the result.
4. After the “Status” has become “Finished”, you will see your accuracy result in “Score” (sometimes this appears empty, and you need to refresh the webpage). Then, click “Add to leaderboard” to send your results to the leaderboard.

Rules and remarks of the competition

1. You will only be able to submit your predictions during given time intervals and the number of submissions that you can make will be limited. The details (and possibly additional rules) will be stated on the CodaBench website.
2. You are limited to a maximum of 1 submission per day and 17 submissions in total during the competition.
3. It is very important that you precisely follow the required construction (e.g., filename and formatting) of the submission file `predictions.txt` and `predictions.zip`. Otherwise, your results will not be computed.
4. The metric “accuracy” alone does not give the complete picture of how well your model is performing, in particular when there are multiple classes. Although the leaderboard is based on accuracy, your report should contain a comprehensive analysis of your classifier (e.g., with a confusion matrix and F1 score).

9 Project Seminar

In addition to the written report, the project will also be examined during a project seminar. During the seminar, groups aiming for grade 5 will present their solutions. The presentations should focus on explaining the methodological ideas and experimental design **used to address the advanced objectives of the project**. It can be assumed that all participants at the seminar are well aquatinted with, at least, the basic version of the project. Furthermore, the presentation should primarily focus on the methodology, rather than purely the empirical results/leaderboard performance.

For these groups, the presentation will also serve as an oral examination of the advanced components of the project. Students should therefore be prepared to explain their methodological choices, experimental setup, and results.

Participation in the project seminar is **mandatory** for everyone.

10 Important Dates

Table 3: Important dates for the project.

Date	Event	Description
2026-04-10	Group signup	Sign up to a group in Lisam, max 4 students per group.
2026-04-28	Leaderboard opens	
2026-05-18	Leaderboard closes	Also submission deadline if you prefer to submit to the leaderboard through Lisam (only required for groups that do not have access to CodaBench).
2026-05-20	Submission of report	Submit through Lisam.
2026-05-22	Project seminar	